

Track 11 — Abo-Lebenszyklus, Anbieter-Onboarding, erfundene Zusagen

Datum: 2026-08-28 **Ausgangspunkt:** 1843542 (Track 10), 1032 Tests **Ergebnis:** 1083 Tests (+51), Typecheck grün, Lint 0 Fehler

Kurzfassung

Fünf Befunde, alle live wirksam, alle von einer grünen Testsuite gedeckt:

#	Schwere	Befund	Wirkung
1	P0	Abo-Lebenszyklus war eine Einbahnstraße	Wer kündigte, behielt Premium/Gold unbegrenzt
2	P1	Anbieter-Registrierung erzeugte ein Konto ohne benutzbares Passwort	Kein registrierter Anbieter konnte sich je anmelden
3	P1	Vermietungs-Angaben und IBAN aus dem Formular wurden weggeworfen	„Stuhlmiete 45 €/Tag“ stand in der Zusammenfassung, nicht in der Datenbank
4	P1	Zwei Oberflächen bewarben Rabattcodes, die der Server nicht kennt	Willkommensmail (WELCOME10) und Startseite (CHAIR2026)
5	P2	Admin-KPI machte jeden Abfragefehler zu einer 0; DAU/WAU zählten Anmeldevorgänge statt Personen	Cockpit-Zahlen nicht von echten Nullbeständen unterscheidbar

1 — P0: Der Abo-Lebenszyklus endete beim Checkout

Befund

`src/app/api/stripe/webhook/route.ts` schaltete beim `checkout.session.completed` die Stufe auf `salons.subscription_tier` frei. Danach kam nichts mehr an.

a) Die Kündigung fand nie ein Profil. `customer.subscription.deleted` suchte den Nutzer über `profiles.stripe_customer_id`:

```
.from('profiles').select('id').eq('stripe_customer_id', customerId)
```

Diese Spalte existiert live (Migration 20260317, Spaltensonde 2026-08-27 bestätigt) — **beschrieben hat sie im gesamten Produktivcode niemand.** Der einzige Schreibzugriff im Repo stand in einer Testdatei:

```
// src/__tests__/e2e/payment-flow.test.ts, vor diesem Track
db().row('profiles', IDS.owner)!.stripe_customer_id = 'cus_test_owner'
```

Der zugehörige Test „stuft bei gekündigtem Abo auf starter zurück“ war grün, weil er den Wert selbst gesetzt hat. In Produktion war die Abfrage garantiert leer: **eine Kündigung stufte nie zurück.**

b) `customer.subscription.updated` gab es nicht. Damit schlugen nicht durch: ein Stufenwechsel im Stripe-Kundenportal (Gold → Premium zeigte weiter Gold), das Ende einer Mahnkette (`unpaid`), ein pausiertes Abo.

c) `invoice.payment_failed` schrieb nur ein Audit-Log. Der Kommentar daneben sagte „Find user by Stripe customer ID and downgrade“ — heruntergestuft wurde nichts. (Das ist an dieser Stelle auch richtig; siehe Fix.)

d) Der Abo-Zweig war der einzige der vier Checkout-Zweige ohne `payment_status` -Prüfung. Termin, Bestellung und Miete prüfen sie seit Track 8. Eine noch nicht eingezogene SEPA-Lastschrift schaltete sofort frei.

Fix

- `createSubscriptionCheckout` setzt jetzt `subscription_data.metadata (user_id, tier, type)` **am Abo**, nicht nur an der Session. Jedes `customer.subscription.*`-Ereignis trägt den Eigentümer damit selbst mit — ohne Umweg über eine Spalte.
- `profiles.stripe_customer_id` wird beim Checkout geschrieben (auch bei noch offener Zahlung: es ist der Rückfall für Altbestand).
- Neuer Handler für `customer.subscription.created/updated/deleted`. Auflösungsreihenfolge: Metadaten → `stripe_customer_id`.
- Die Stufe kommt aus der **tatsächlich gebuchten Price-ID**, nicht aus `metadata.tier` — bei einem Wechsel im Kundenportal steht dort noch die Stufe der ursprünglichen Buchung. Neues Modul `src/lib/subscription-tier.ts`.
- Gehört eine Price-ID zu keiner konfigurierten Stufe, wird **nichts** umgestellt (`subscription_price_unknown` im Audit-Log). Die Platzhalter aus `SUBSCRIPTION_PRICES` (`price_gold` ...) sind von der Zuordnung ausgenommen: ohne diesen Riegel bekäme in einer Umgebung ohne `STRIPE_PRICE_*` jedes Abo mit passendem Platzhalter-String die falsche Stufe.
- Statusbewertung: `active / trialing` → freischalten;
`canceled / unpaid / incomplete_expired / paused` → auf `starter`;
`past_due / incomplete / Unbekanntes` → **Stufe unverändert**, Audit-Eintrag `subscription_grace`. Eine Rückstufung beim ersten fehlgeschlagenen Einzug würde jeden Anbieter treffen, dessen Karte einmal abgelehnt wird — Stripe mahnt danach noch mehrfach.
- `invoice.payment_failed` protokolliert weiterhin und **warnt den Anbieter jetzt zusätzlich per In-App-Benachrichtigung**. Die Rückstufung bleibt am Ende der Mahnkette.
- Ein `tier` außerhalb der drei Stufen landet nicht mehr in der Datenbank.

Belegt durch

`src/__tests__/e2e/payment-flow.test.ts` (9 neue Fälle) und `src/lib/__tests__/subscription-tier.test.ts` (11 Fälle). Der Kernfall: `kuendigt ohne stripe_customer_id` — ueber die Metadaten am Abo, also genau der Produktionszustand, in dem die Kündigung bisher wirkungslos war.

2 — P1: Das Anbieter-Konto war nicht benutzbar

Befund

`POST /api/register-provider` legte ein Supabase-Auth-Konto mit einem Zufallspasswort an:

```
// Send welcome email with temp password (do not return password in JSON)
const { sendWelcomeEmail } = await import('@lib/email')
await sendWelcomeEmail(d.em, `${d.vn} ${d.nn}`)
```

`sendWelcomeEmail(to, name)` nimmt **kein Passwort entgegen**. Das Passwort wurde nicht zurückgegeben (richtig), nicht gespeichert (richtig) — und auch nicht verschickt. Die Bestätigungsseite sagte „Prüfung innerhalb 24h. E-Mail-Bestätigung folgt.“ und schwieg zum Passwort. Der Login läuft über `supabase.auth.signInWithPassword (auth.config.ts)`, es gab also genau einen Weg hinein — „Passwort vergessen“ — und keinen Hinweis darauf.

Fix

- Die Registrierung löst selbst `resetPasswordForEmail` aus (derselbe Supabase-Weg wie `/api/auth/forgot-password`).
- Die Antwort meldet `passwordEmailSent`; die Bestätigungsseite sagt entweder „Wir haben dir eine E-Mail geschickt, mit der du dein Passwort festlegst“ oder — wenn der Versand scheiterte — dass er scheiterte, mit Link auf „Passwort vergessen“. Kein stiller Fehlschlag.
- Neue Anbieter-Begrüßungsmail `sendProviderWelcomeEmail` mit den drei echten nächsten Schritten. Vorher bekamen Anbieter die Kundenmail („Salon entdecken → Termin buchen → Beauty genießen“).

3 — P1: Erfragte Angaben landeten nirgends

Befund

Schritt 3 des Formulars fragt „Ja, ich vermiete Stühle“ plus Preis/Tag; die Zusammenfassung zeigt „Stuhlmiete 45 €/Tag“. Die Route nahm `chair` und `cpr` entgegen und schrieb keins von beidem. `salons.chair_rental` (boolean) und `salons.chair_price_day` (numeric, Euro) existieren live (Spaltensonde 2026-08-27) und blieben leer.

Dasselbe für die IBAN: im Schema validiert, nie verwendet. In `OnboardingGate` (der zweite Registrierungsweg) landete sie zusätzlich in `localStorage` unter `cm_setup_profile` — Bankdaten ausschließlich im Browser, mit einem Formular, das aussah, als wären sie hinterlegt.

Fix

- `chair_rental` und `chair_price_day` werden geschrieben. `parseDayPrice (src/lib/provider-registration.ts)` akzeptiert Komma als Dezimaltrennzeichen und gibt `null` zurück, wenn nichts Verwertbares dasteht — **kein erfundener Standardpreis**. Deckel bei 10.000 € fängt den Cent/Euro-Vertipper („35000“ statt „350“) ab, bevor er auf einer öffentlichen Salonseite landet.
- Das IBAN-Feld ist aus **beiden** Formularen entfernt und wird von der Route nicht mehr angenommen. Auszahlungsdaten gehören nach der Anmeldung in `payout_accounts` über `/api/me/payout-`

`account` — dort verlässt die volle IBAN den Server nie wieder, herausgegeben werden nur die letzten vier Stellen.

- `OnboardingGate` schickte `gb: true` (Gewerbeschein vorhanden) fest verdrahtet, obwohl dieser Ablauf danach gar nicht fragt. Jetzt `false`.
- **AGB- und Datenschutz-Einwilligung werden protokolliert** (`audit_logs`, Aktion `provider_registration_consent`, mit IP). Bewusst **nicht** in `consents`: diese Tabelle hängt live an einer Buchung (`booking_id NOT NULL`) und ist für den Behandlungs-Consent gedacht — ein Insert wäre in 23502 gelaufen.
- **Rate-Limit** ergänzt (5/Std pro IP, 3/Std pro Adresse). Der Endpunkt legt pro Aufruf ein Auth-Konto, ein Profil, einen Salon an und verschickt zwei Mails; er hatte keinerlei Begrenzung.
- Scheiterte der Salon-Insert, blieben Auth-Konto und Profil zurück und die Adresse war für jeden weiteren Versuch verbrannt. Der Fehlerfall räumt das Profil jetzt ab.

Belegt durch

`src/__tests__/e2e/provider-onboarding.test.ts` (15 Fälle), darunter `nimmt keine IBAN mehr entgegen` (prüft, dass die Nummer in **keiner** Tabelle auftaucht) und `laesst kein Profil ohne Salon zurueck`.

4 — P1: Zwei Zusagen, die der Server nicht einlöst

Befund

Track 9 hat `PROMO_CODES` aus `src/lib/constants.ts` entfernt: `CHAIR2026` (15 %), `WELCOME10` (10 %), `BEAUTY5` (5 €) waren eine reine Browser-Konstante. Der Server kennt sie nicht — er prüft die Tabelle `promo_codes` und belegt dort ein Kontingent (`claimPromoCode`).

Die Konstante war weg, das Versprechen nicht:

1. `src/lib/email.ts` — die Willkommensmail an **jeden** neu registrierten Nutzer: „Nutze den Code **WELCOME10** für 10% auf deine erste Buchung!“ Eine E-Mail ist die haltbarste Form dieser Zusage: sie liegt im Postfach, wenn der Code beim Buchen abgewiesen wird.
2. `src/components/HomeClient.tsx` — der Promo-Banner der **Startseite**: „Code: CHAIR2026“, fest im Quelltext, für jeden Besucher.

Der Prozentsatz im Banner ist echt (`offers.discount_percent`, höchstes aktives Angebot). Ein Code gehört nicht dazu: `offers` hat live **keine** Spalte `code` oder `promo_code` (Spaltensonde 2026-08-28).

Fix

Beide Zusagen entfernt. Der Banner führt zu `/offers`, statt eine Eingabe zu versprechen, die niemand einlöst.

Belegt durch

`src/__tests__/keine-erfundenen-zusagen.test.ts` — Quelltext-Wächter über `src/`. Zusätzlich eine breitere Regel für `email.ts`: **jedes** Wort aus Großbuchstaben mit angehängter Zahl (`/\b[A-Z]`

{4,}\d{1,3}\b/) lässt den Build rot werden. Wer einen Rabatt bewerben will, legt ihn in `promo_codes` an und lädt ihn — dann steht er nicht im Quelltext.

5 — P2: Das KPI-Cockpit maß, was es nicht messen konnte

Befund

a) Jeder Fehler wurde zu einer 0. `safeCount` in `src/app/api/admin/kpi/route.ts` fing alles ab und gab 0 zurück — eine fehlende Tabelle, ein Rechtefehler, ein Timeout. „Buchungen 30d: 0“ war nicht davon zu unterscheiden, dass wirklich niemand gebucht hat. Dieselbe Verwechslung hatte Track 10 im Anbieter-Dashboard.

b) DAU und WAU zählten Anmeldevorgänge, nicht Personen.

```
// DAU (Daily Active Users) – User mit Login in den letzten 24h
dau = await safeCount('login_attempts', q => q.eq('success', true)...)

```

Das ist die Zahl der **Zeilen**. Wer sich an einem Tag von Handy und Rechner anmeldet, zählte zweimal. DAU war systematisch zu hoch, und `dau_wau_ratio` — die Zahl, an der Stickiness gemessen wird — entstand aus zwei verschieden stark überzeichneten Werten. `login_attempts` hat live **keine** `user_id` (Spaltensonde 2026-08-27); die Person steckt nur in `email`.

Fix

- `safeCount` gibt bei einem Fehler `null` zurück und trägt den Grund in ein `errors`-Feld der Antwort ein. Jede daraus abgeleitete Quote wird ebenfalls `null`. Die Cockpit-Seite zeigt dafür „—“, lässt den Meilenstein-Balken leer (ein Balken auf 0 % behauptet einen Stand) und listet die Fehlschläge sichtbar auf.
- DAU/WAU zählen eindeutige, kleingeschriebene Adressen mit erfolgreicher Anmeldung im Fenster. Obergrenze 20.000 Rohzeilen; wird sie erreicht, meldet die Antwort `capped: true` und die Seite sagt, dass die Werte Untergrenzen sind — statt eine zu kleine Zahl als Wahrheit auszugeben.
- Die Unterzeilen im Cockpit hießen „(Logins 24h)“. Jetzt „Personen, 24h“ — und das stimmt auch.

Belegt durch

`src/__tests__/admin-kpi-unbekannt.test.ts` (11 Fälle).

Live-Schema-Sonde 2026-08-27/28

Alle geschriebenen Spalten wurden vor der Implementierung gegen die Produktionsdatenbank geprüft (PostgREST-Spaltenprobe, ANON-Key, nur lesend):

Spalte	Befund
<code>profiles.stripe_customer_id</code>	vorhanden
<code>salons.chair_rental</code> , <code>salons.chair_price_day</code>	vorhanden (<code>numeric</code> , Euro — kein <code>_cents</code>)
<code>salons.subscription_tier</code>	vorhanden
<code>salons.stripe_subscription_id</code>	fehlt → Abo-ID wird nicht am Salon abgelegt, die Zuordnung läuft über Stripe-Metadaten
<code>offers.code</code> / <code>offers.promo_code</code>	fehlt → es gibt keinen echten Code zum Anzeigen
<code>login_attempts.user_id</code>	fehlt → DAU über <code>email</code>
<code>consents.consent_type</code>	fehlt ; <code>booking_id</code> ist NOT NULL → Registrierungs-Consent geht in <code>audit_logs</code>

`src/test/live-schema.ts` ist um `profiles.stripe_customer_id`, `profiles.phone`, `salons.subscription_tier`, `salons.is_verified`, `salons.chair_rental` und `salons.chair_price_day` ergänzt.

Offen — bewusst nicht in diesem Track

payout_accounts ist für **anon** lesbar. Die Sonde vom 2026-08-28 liefert `[]` statt `42501`: die Tabelle ist leer, aber die Rolle **anon** darf sie lesen. Sobald ein Anbieter Auszahlungsdaten hinterlegt, liegen `user_id`, `account_holder` und `iban_last4` öffentlich. Das ist der bekannte Befund „anon-GRANTS offen“; die Gegenmigration `supabase/migrations/20260827_anon_grant_lockdown.sql` liegt bereit und ist **nicht angewendet** — Agents haben keinen DB-Zugang und es gibt keinen Migrations-Runner.

Das ist mit ein Grund, warum die Registrierungs-IBAN in diesem Track **nicht** in `payout_accounts` geschrieben wird: das hätte jedem neuen Anbieter sofort eine öffentlich lesbare Zeile verschafft. Der bestehende Schreibweg über `/api/me/payout-account` bleibt unverändert.

Termin-Erinnerungen. `sendBookingReminder` existiert, wird aber von keinem Cron aufgerufen (`vercel.json` kennt drei Crons, keiner davon). Die Buchungsbestätigungsmail verspricht keine Erinnerung, es ist also keine falsche Zusage — aber eine unfertige Funktion.

Verifikation

```
npx tsc --noEmit      → 0 Fehler
npm run lint          → 0 Fehler (19 Bestandswarnungen)
npx vitest run        → 59 Dateien, 1083 Tests, alle grün (vorher 1032)
```

`npm run build` schlägt **lokal** mit `SUPABASE_SERVICE_ROLE_KEY` fehlt beim Prerender von `/shop` fehl. Das ist Umgebung, nicht Code: die Variable steht nicht in `.env.local` (bekannter Zustand, siehe `docs/SUPABASE_SERVICE_ROLE_KEY_STATUS.md`), auf Vercel ist sie gesetzt. Kein Pfad dieses Tracks ist beteiligt.

Geänderte Dateien

Neu: `src/lib/subscription-tier.ts`, `src/lib/provider-registration.ts`,
`src/__tests__/e2e/provider-onboarding.test.ts`, `src/__tests__/admin-kpi-unbekannt.test.ts`,
`src/__tests__/keine-erfundenen-zusagen.test.ts`, `src/lib/__tests__/subscription-`
`tier.test.ts`

Geändert: `src/lib/stripe.ts`, `src/app/api/stripe/webhook/route.ts`, `src/app/api/register-`
`provider/route.ts`, `src/app/(public)/register/anbieter/page.tsx`,
`src/components/OnboardingGate.tsx`, `src/components/HomeClient.tsx`, `src/lib/email.ts`,
`src/app/api/admin/kpi/route.ts`, `src/app/(admin)/admin/super/kpi/page.tsx`, `src/test/live-`
`schema.ts`, `src/__tests__/e2e/payment-flow.test.ts`